AI CLIENT HUNTER

Complete Guide: Architecture, Workflow and Phase-by-Phase Build Plan

A local-first lead research and sales-assistance system. It finds business and project leads from **free, permitted sources**, verifies them, scores them, stores them in a MySQL CRM and prepares personalized message drafts. **Nothing is sent without your approval.**

Version	1.2 (v1.1 plus full setup steps for every phase; database provider and job queue corrected after checking current packages)
Date	October 2026
Stack	React (Vite), ASP.NET Core Web API, MySQL Community, built-in background worker, Ollama (local AI)
Cost target	Rs. 0 for APIs and licences. Hosting is separate and only needed after local validation.
Build path	Windows laptop first, deploy online after testing

Contents

Part 1: Plan	1 Overview, 2 Lead types and scoring, 3 Free sources, 4 Architecture and stack, 5 Workflow, 6 Database, 7 Safe rules, 8 Phases, 9 Testing plan, 10 Budget, 11 Risks
Part 2: Build guide	Phase 0 Source test, 1 Local setup, 2 MySQL, 3 CRM basics, 4 Search jobs, 5 OpenStreetMap source, 6 Verify, dedupe and score, 7 AI with Ollama, 8 Drafts and approval, 9 Testing and security, 10 Deployment
Appendices	A Troubleshooting, B Master checklist

Part 1 explains what the system is and why it is designed this way. Part 2 is the step-by-step build guide for every phase.

1. Project Overview

AI Client Hunter is a lead research and sales workflow tool. It helps you find people and businesses that may need a website, mobile app, software or automation, and manage them in one place. It is a **research assistant with a human in the loop**, not an unlimited scraper.

What it does

- Finds businesses whose website is missing or not found in the checked source (OpenStreetMap listings).
- Finds public posts where someone asks for a website, app or developer (official APIs only).
- Spots new-business signals (new listings, new startups) that may point to a need for a digital presence.
- Verifies, deduplicates and scores each lead as Hot, Warm or Potential, and keeps the source link as evidence.
- Drafts a personalized message for each lead. You review, edit and approve it before anything is sent.
- Tracks follow-ups and outcomes in a CRM, with a do-not-contact list.

What it does not do

- **It cannot guarantee 500 leads a day.** Free sources give roughly 20 to 50 verified leads a day. Treat 500 as a long-term goal only.
- A business with no website listed is only a *potential* prospect. It does not prove the business wants a website.
- It does not scrape Google Maps, Google Search, LinkedIn, Upwork or Facebook. Those platforms forbid automated scraping in their terms, and doing so risks account and IP bans (see Section 3).
- It does not bypass logins, CAPTCHAs or rate limits, and it does not auto-send unsolicited messages.

2. Lead Types and Priority

Type / priority	Meaning	Example signal	Action
Hot: project seeker	An explicit public request for a developer or a quote.	"Need a website developer for my shop."	Review quickly and send a tailored draft.
Warm: new business	A launch, opening or startup signal suggests a possible need.	New clinic listed; newly registered startup.	Research the need, then personalize.
Potential: no website	Listing has no website field, or one could not be verified.	Phone and address present, no site URL.	Verify manually before outreach.

Initial scoring rules (draft, tune after real data)

Points are added for each signal found. Every score is saved with its reasons and source links, so you can always see why a lead got its priority.

Signal	Points
Public post clearly asks for a website, app or software developer	+60
Post mentions budget, deadline or detailed requirements	+10
Post or listing is from the last 7 days	+10
New-business signal (recent listing, new startup, opening announcement)	+25
Website field missing in the source listing	+15
Website checked: no live site found (DNS and HTTP both failed)	+10
Phone number or business contact available from a permitted source	+5
Located in a target city or state	+5

Priority	Rule
Hot	Score 60 or more **and** an explicit project request exists.
Warm	Score 30 to 59.
Potential	Score below 30, or website missing with no other signal.

> The score is a decision aid, not a prediction of purchase. Never mark a lead Hot unless a public request is actually present in the evidence.

3. Lead Sources: Free and Permitted Only

The whole project depends on which sources you can use legally and reliably. This version uses only free sources and official APIs. Check each source's current terms and limits before coding against it, because they change.

Sources to use

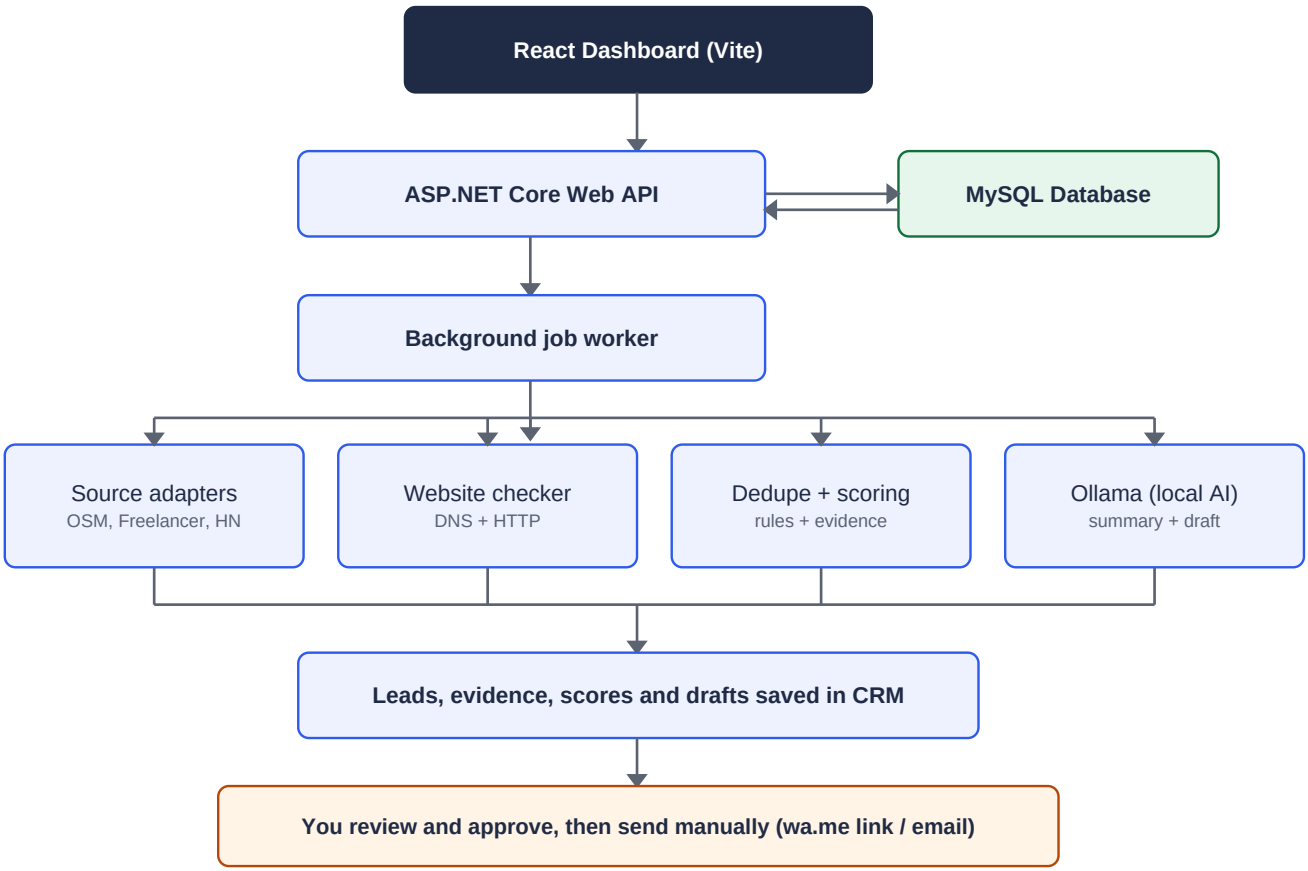
Lead type	Source	Notes
No website / new business	OpenStreetMap: Overpass API or Geofabrik India data extract	Best free option. Businesses have phone, address and website tags. Bulk download into MySQL avoids API quota issues. Requires ODbL attribution.
No website / new business	data.gov.in open datasets	Some datasets list registered businesses. Coverage and licence differ per dataset.
New business	Startup India recognised-startup lists; public domain registration age (RDAP)	Warm signals only. Check usage terms.
Project seekers	Freelancer.com official API	Searches projects with an API token. Check current terms and limits.
Project seekers	Hacker News (Algolia API); RemoteOK API; GitHub API	Free and permitted, but mostly global and tech-focused.
Project seekers	Reddit API (for example hiring subreddits)	Free tier is for non-commercial use. Read the terms before using it for business.

Sources not used, and the alternative

Platform	Why not	Alternative
Google Maps (scraping)	Scraping is against Google's terms. Risk of IP or account block.	OpenStreetMap for free bulk data. Google Places API (official) only later, with a hard spending cap, if you accept billing.
Google Search (to check websites)	Scraping results is not permitted.	Own DNS and HTTP checker, plus the website field from the source listing.
LinkedIn posts	No public post-search API. Scraping leads to account bans.	Search manually on LinkedIn yourself and add promising leads to the CRM by hand.
Upwork	Scraping is forbidden. Official API is for approved partners only.	Use Upwork's own job alerts by email, and add leads manually.
JustDial, IndiaMART, Facebook groups	No free official API. Scraping is against their terms.	Manual research; add leads by hand.

Expect fewer Hot leads. The platforms where people post "I need a website" are the ones that block automation. Most volume will be Warm and Potential leads from OpenStreetMap. This is the honest trade-off of a free, compliant setup.

4. System Architecture



Technology stack

Layer	Technology	Role	Cost
Frontend	React.js (Vite)	Dashboard, filters, lead detail, approval buttons, export	Rs. 0
Backend	ASP.NET Core Web API (.NET 10 LTS)	Business logic, endpoints, authentication, audit log	Rs. 0
Database	MySQL Community + EF Core (Microting MySQL provider 10.0.x)	Leads, jobs, evidence, drafts, CRM	Rs. 0
Background jobs	Built-in BackgroundService + queue in MySQL	Runs searches in the background; jobs saved in search_jobs so they survive restarts	Rs. 0
Local AI	Ollama (3B to 8B model)	Classification, summaries, message drafts, JSON output	Rs. 0
Website check	Own C# DNS and HTTP checker	Verified / not found / unknown	Rs. 0
Export	CSV / Excel library	Download reviewed leads	Rs. 0

Changed in v1.2: Hangfire is replaced by a built-in background worker, because its MySQL storage packages have not been updated since 2020. The MySQL provider is Microting.EntityFrameworkCore.MySql 10.0.x (EF Core 10), because the stable Pomelo release still targets EF Core 9.

AI behind an interface

The backend calls AI through one interface (for example `IAiProvider`). Today it is implemented by Ollama. If you later want a hosted model, only configuration and one new class change, not the rest of the code. Hosted models (Claude on Vertex AI, Gemini) are paid or limited, so they stay optional and off by default.

5. End-to-End Workflow

#	Step	What happens
1	Command	You enter a goal in the dashboard, for example: *Ahmedabad, clinics, website missing, limit 50*.
2	Validate and queue	The API validates the input and saves a Search Job (status Queued). A background worker runs it so the UI never freezes.
3	Collect	Source adapters fetch records within each source's limits. Every record keeps its source URL and retrieval time.
4	Normalize	Names, phone numbers (+91 format), city and category are converted to one standard format.
5	Deduplicate	Same business again? Merge the evidence into the existing lead. Matching uses domain, source ID, and business name plus city.
6	Verify website	If the listing has no website, try likely domains with DNS and HTTP checks. Result is only Verified, Not found or Unknown. No guessing.
7	Score	Apply the rules in Section 2. Save the points, reasons and source links.
8	AI analysis	Ollama receives only the collected evidence and returns structured JSON: summary, recommended service, draft message. It must not invent facts.
9	Save	Leads, evidence, scores and drafts are stored in MySQL. The job outcome and any errors are logged.
10	Review	The dashboard shows results with priority, reason, website status and source links. Filter and export as needed.
11	Approve	You read the draft, edit it and press Approve. Without approval nothing goes out.
12	Send and track	You send manually using a WhatsApp click-to-chat link or your own email. Then log the status: Contacted, Replied, Follow-up due, Won or Lost.

CRM status life cycle

New, Needs review, Qualified, Draft ready, Approved, Contacted, Replied, Follow-up due, then **Won** or **Lost**. At any point a lead can be moved to **Do not contact**, and it will never appear in an outreach queue again.

6. Database Design (MySQL)

Table	Purpose
users	App accounts and roles.
search_jobs	Command, filters, sources used, status, timestamps, result counts.
source_records	Raw and normalized source data: source URL or ID, retrieved time, permitted metadata.
leads	One canonical business or project lead with its current CRM state.
lead_evidence	Links a lead to its source records, with evidence notes.
lead_scores	Priority, points, reasons, rule version, timestamp.
message_drafts	Draft versions, status, approver and approval time.
crm_activities	Calls, messages, replies, follow-up tasks and outcomes.
do_not_contact	Opt-out and suppression records. Kept even if a lead is deleted.
audit_logs	Important actions: exports, approvals, sends, logins.

Key lead fields

Group	Fields
Identity	Lead ID, business or project name, category, city, state, country.
Website	URL, status (verified / not found / unknown), last checked time, check result.
Evidence	Source name, source URL, date found, short factual summary, retrieval timestamp.
Contact	Business contact only where permitted, with its source, plus the do-not-contact flag.
Qualification	Lead type, priority, score, reasons, confidence, relevant service.
AI output	Requirement summary, recommended service, draft, draft version, approval record.

7. Safe Operating Rules

- Use only official APIs, open data and permitted sources. No login-wall scraping, no CAPTCHA bypass, no identity rotation.
- Every lead has a source and a retrieval date. If something cannot be verified, label it **unknown**.
- AI drafts are suggestions. A human approves before anything is sent.
- Do not mass-send unsolicited messages. Keep volumes small, relevant, and honor opt-outs immediately.
- Keep API keys and passwords in environment variables or user-secrets. Never in frontend code or a public repository.
- Protect the CRM with login, roles, backups and audit logs. Restrict exports to authorized users.
- Respect source licences: show OpenStreetMap attribution and follow each source's retention rules.

Data protection (India)

Business details can include personal data of proprietors, which falls under the Digital Personal Data Protection Act, 2023. Before going beyond a private prototype, define: why you hold each field, how long you keep it, how a person can ask for deletion, and how the suppression list survives deletion. This is not legal advice; confirm requirements with a qualified professional.

Outreach channels

The first version uses **manual sending only** (WhatsApp click-to-chat link or your own email). This avoids WhatsApp Business API approvals and SMS DLT registration costs. If you automate later, WhatsApp needs approved templates and opt-in, SMS needs DLT registration, and bulk email needs consent and deliverability care.

8. Development Phases

Phase	Deliverable	Completion check
0. Source spike (new)	Test OpenStreetMap and Freelancer API on real queries. Note field quality, limits, licence.	You know what each source really returns.
1. Local setup (done)	React app and ASP.NET Core API in one repository.	Both start on the laptop and React shows API status.
2. MySQL	Schema, EF Core migrations, connection config, sample data.	Create, read, update tests pass.
3. CRM basics	Lead list, detail page, filters, statuses, CSV export.	Manual sample leads work end to end.
4. Search jobs	Command form, background worker, source adapter interface.	Jobs are tracked and errors are visible.
5. First source	OpenStreetMap adapter (Overpass or Geofabrik data).	Results carry source URLs and timestamps.
6. Verify, dedupe, score	Website checker, normalization, merge logic, rule-based scoring.	Duplicate and unknown cases tested.
7. AI analysis	Ollama integration with structured JSON output and rationale.	AI does not invent facts in test cases.
8. Message drafts	Draft generation and approval queue.	Sending stays disabled until approved.
9. Full testing	Automated and manual checklist, bug fixes.	All critical tests pass locally.
10. Deployment	Secure hosting, production DB, HTTPS, backups, monitoring.	Smoke test and rollback plan complete.

Recommended MVP: one source (OpenStreetMap), one lead type (website missing), one city (Ahmedabad), draft-only. Target 20 to 50 unique, valid leads a day. Expand to more sources only after this works.

9. Testing Plan

Area	Pass condition
Command validation	Empty or invalid searches are rejected with a clear message.
Source adapter	Respects quota and rate limits and logs errors.
Deduplication	Repeated results merge instead of creating duplicates.
Website verification	Correctly separates verified, not found and unknown.
AI output	Classification has a reason and invents no facts outside the evidence.
MySQL persistence	Data and relationships survive an application restart.
CRM filters and export	Filters match records; exports include source and status.
Approval flow	No message can be sent before approval.
Privacy and opt-out	Do-not-contact records never appear in outreach queues.
Security	No secrets exposed; endpoints require correct authentication.
Performance	Background jobs do not freeze the UI; failed jobs retry safely.

10. Budget

Item	Local prototype	Later (online)
React, ASP.NET Core, MySQL	Rs. 0	Rs. 0 licence cost; hosting extra
Data sources (OSM, Freelancer, HN, others)	Rs. 0	Rs. 0 while staying within free limits
Ollama local AI	Rs. 0 (uses laptop power)	Can stay local or add a hosted model
Hosted AI (Claude on Vertex AI, Gemini)	Not used	Optional, paid per token. Set a budget alert and spending cap first.
Hosting, domain, backups	Not needed locally	Depends on provider and traffic
WhatsApp / SMS / email provider	Not needed (manual sending)	Provider fees and compliance if you automate

No monthly total is promised. Build the small local prototype first, measure real results, and pay for a service only where it clearly adds value.

11. Realistic Expectations and Risks

Risk	What to expect	Mitigation
Low volume from free sources	20 to 50 verified leads a day, not 500.	Measure real numbers; add sources one at a time.
Missing website tag is often just an incomplete listing	Many false positives.	Label as Potential; verify before contacting.
Few Hot leads	Platforms with explicit requests block automation.	Add manual leads from LinkedIn and Upwork alerts.
Small local AI model errors	May summarize badly or invent details.	Strict JSON schema, evidence-only prompts, human review.
Source terms change	Free limits and licences can change.	Re-check terms before each integration.
Public Overpass server limits	Guidance is about 10,000 requests and 1 GB a day; heavy users are throttled.	One query per job, 24-hour cache, Geofabrik extract for bulk.
Privacy compliance	Contact data can be personal data.	Retention policy, deletion process, do-not-contact list.

Laptop requirements

Windows 10 or 11, minimum 8 GB RAM (16 GB recommended for Ollama). Install .NET SDK, Node.js, MySQL Community, Ollama and Git. A 3B to 8B parameter model is enough; speed depends on your hardware.

Part 2: Phase-by-Phase Build Guide

This part walks through every phase, from the source test (Phase 0) to deployment (Phase 10). Each phase has the same layout: goal, what to install, numbered steps with commands and key code, and a completion check. Do not start a phase until the previous completion check passes.

How to use this guide

- Work in order. Commit to git at the end of each phase (`git add . && git commit -m "Phase N done"`).
- Code in this PDF is **reference code**: it shows the structure and the important parts. It was not compiled in this document, so when you reach a phase, ask Claude for the complete files of that phase, run `dotnet build`, and paste any error back to fix it.
- Keep all secrets (database password, JWT key, future API tokens) in user-secrets or environment variables. Never in the frontend or in git.
- Windows commands use PowerShell. Folder names follow the Phase 1 project.

Corrections in v1.2 (checked in October 2026): (1) The stable Pomelo MySQL provider still targets EF Core 9. Because the project uses .NET 10, this guide uses **Microting.EntityFrameworkCore.MySql 10.0.x**, which targets EF Core 10 and is built from the Pomelo work. If you prefer Pomelo itself, use .NET 8 or 9 with Pomelo 9.0.0. (2) Hangfire's MySQL storage packages have not been updated since 2020, so this guide uses the built-in **BackgroundService** with jobs saved in the search_jobs table. (3) The public Overpass server asks users to stay near 10,000 requests and 1 GB per day, so the app makes one small query per job and caches results.

Final folder layout (after all phases)

```
AiClientHunter/
  server/
    AiClientHunter.Api/
      Program.cs
      Domain/        Entities.cs
      Data/          AppDbContext.cs, DbSeeder.cs
      Endpoints/     LeadEndpoints.cs, JobEndpoints.cs, DraftEndpoints.cs, AuthEndpoints.cs
      Jobs/          JobQueue.cs, JobWorker.cs
      Sources/       ISourceAdapter.cs, OsmOverpassAdapter.cs, OsmMaps.cs
      Services/      Normalizer.cs, WebsiteChecker.cs, DedupeService.cs, Scorer.cs, Suppression.cs
      Ai/            IAiProvider.cs, OllamaProvider.cs
    AiClientHunter.Tests/
  client/
    src/  api.js, App.jsx, pages/ (LeadList, LeadDetail, SearchJobs, ApprovalQueue, Login)
  README.md
```

Phase 0: Source Test (no app code)

Goal	Learn what the free sources really return before building anything on top of them.
Time (rough)	1 to 2 hours

Before you start

- A browser and a notepad or spreadsheet. No install needed.
- Decide your first city and category (suggested: Ahmedabad, clinics).

Step 1: Run an OpenStreetMap query by hand

Open `overpass-turbo.eu`, paste the query below and press Run. The bbox is south, west, north, east. These numbers roughly cover Ahmedabad; adjust after looking at the map.

```
[out:json][timeout:60][bbox:22.90,72.45,23.15,72.75];
(
  nwr["amenity"~"^(clinic|doctors|dentist)$"];
);
out center tags;
```

Step 2: Measure the data quality

Use Export, then Raw data, and copy the JSON into a spreadsheet or count it with the Data tab. Fill in this table.

Metric	Your result
Total places returned	
Places with a name	
Places with a phone (phone or contact:phone)	
Places with a website (website, contact:website or url)	
Places with name and phone but no website	

Step 3: Estimate the false-positive rate

Pick 20 places that have no website tag. Search each name by hand in a normal browser. Note how many actually have a website. This tells you how much to trust the "website missing" signal. Do this manually and only for 20 records.

Step 4: Try the project-seeker sources

Freelancer.com: create a developer account and app at the Freelancer developer site, get a token, and run a project search for "website". Note which fields come back (title, description, budget, date, link), the rate limits, and whether commercial lead-gen use is allowed. If your account cannot get API access, mark the source as dropped.
Hacker News: open `https://hn.algolia.com/api/v1/search_by_date?query=website%20developer` in a browser and see how many results are relevant to India. Expect few.

Step 5: Decide

For each source write: usable (yes/no), fields available, expected leads per day, terms OK (yes/no/unclear). Keep OpenStreetMap as the first source unless the test shows it is unusable.

Completion check

- [ ] You have numbers for OpenStreetMap (total, with phone, with website).

- [ ] You know the rough false-positive rate for "no website tag".
- [ ] You have a keep or drop decision for each source.

Phase 1: Local Setup (delivered)

Goal	React app and ASP.NET Core API both start on your laptop, and React shows the API status.
Time (rough)	30 to 60 minutes

Before you start

- .NET SDK 10 (LTS). Check: `dotnet --version`. With .NET 8, change net10.0 to net8.0 in the .csproj, but then use Pomelo 9.0.0 in Phase 2 instead of the Microting provider.
- Node.js 22 LTS. Check: `node --version`.
- Git. Check: `git --version`.
- Extract **AiClientHunter-phase1.zip** (delivered earlier in this chat).

Step 1: Start the backend

```
cd server\AiClientHunter.Api
dotnet run
```

Open `http://localhost:5000/api/health`. You should see JSON with status ok.

Step 2: Start the frontend (new terminal)

```
cd client
copy .env.example .env
npm install
npm run dev
```

Open `http://localhost:5173`. The page should show "Connected: AI Client Hunter API (ok)".

Step 3: Put the project in git

```
cd AiClientHunter
git init
git add .
git commit -m "Phase 1: project scaffold"
```

The .gitignore already excludes .env, node_modules, bin and obj.

Completion check

- [ ] dotnet run starts the API and /api/health returns JSON.
- [ ] npm run dev starts React and the page shows Connected.
- [ ] First git commit exists and .env is not in it.

Phase 2: MySQL: Schema, Migrations, Sample Data

Goal	A MySQL database with all core tables created by EF Core migrations, and a create/read/update/delete test that passes.
Time (rough)	2 to 4 hours

Before you start

- Install **MySQL Community Server** and MySQL Workbench with the Windows MySQL Installer. Remember the root password.
- Check the MySQL service is running (Windows Services, name starts with MySQL).
- Packages used: Microting.EntityFrameworkCore.MySql, Microsoft.EntityFrameworkCore.Design, dotnet-ef tool.

Step 1: Create the database and an app user (not root)

Open MySQL Workbench or run `mysql -u root -p`, then run:

```
CREATE DATABASE ai_client_hunter CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'hunter'@'localhost' IDENTIFIED BY 'CHOOSE_A_STRONG_PASSWORD';
GRANT ALL PRIVILEGES ON ai_client_hunter.* TO 'hunter'@'localhost';
FLUSH PRIVILEGES;
```

Step 2: Add packages and store the connection string as a secret

```
cd server\AiClientHunter.Api
dotnet add package Microting.EntityFrameworkCore.MySql
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet tool install --global dotnet-ef
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Default" `
  "Server=localhost;Port=3306;Database=ai_client_hunter;User=hunter;Password=YOUR_PASSWORD"
```

User-secrets keep the password outside the project folder, so it can never be committed by mistake.

Step 3: Create the entities (Domain/Entities.cs)

Below are the two main tables in full. Create the others the same way using the field lists in the table that follows. Indexed text columns **must** have [MaxLength], otherwise MySQL fails with "key was too long".

```
using System.ComponentModel.DataAnnotations;
namespace AiClientHunter.Api.Domain;

public class Lead
{
    public long Id { get; set; }
    [MaxLength(300)] public string Name { get; set; } = "";
    [MaxLength(100)] public string? Category { get; set; }
    [MaxLength(100)] public string? City { get; set; }
    [MaxLength(100)] public string? State { get; set; }
    [MaxLength(60)]  public string Country { get; set; } = "India";
    [MaxLength(500)] public string? WebsiteUrl { get; set; }
    [MaxLength(20)]  public string WebsiteStatus { get; set; } = "unknown"; // verified|not_found|unknown
    public DateTime? WebsiteCheckedAt { get; set; }
    [MaxLength(30)]  public string? Phone { get; set; }                     // +91XXXXXXXXXX
    [MaxLength(20)]  public string LeadType { get; set; } = "potential";    // hot|warm|potential
    public int Score { get; set; }
    [MaxLength(30)]  public string Status { get; set; } = "new";
    [MaxLength(300)] public string? DedupeKey { get; set; }
    public DateTime? FollowUpDueAt { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } = DateTime.UtcNow;
    public List<LeadEvidence> Evidence { get; set; } = new();
}

public class SearchJob
{
    public long Id { get; set; }
    [MaxLength(100)] public string City { get; set; } = "";
```

```
    [MaxLength(100)] public string Category { get; set; } = "";
    [MaxLength(20)]  public string LeadType { get; set; } = "potential";
    public int ResultLimit { get; set; } = 50;
    [MaxLength(20)]  public string Status { get; set; } = "queued"; // queued|running|completed|failed
    public int RecordsFound { get; set; }
    public int NewLeads { get; set; }
    public int Duplicates { get; set; }
    public string? Error { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
    public DateTime? StartedAt { get; set; }
    public DateTime? FinishedAt { get; set; }
}
```

Entity (table)	Fields
SourceRecord	Id, SourceName, SourceId, SourceUrl, RetrievedAt, RawJson. Unique on SourceName + SourceId.
LeadEvidence	Id, LeadId, SourceRecordId, Note, CreatedAt (links a lead to its source records).
LeadScore	Id, LeadId, Points, Priority, Reasons (text), RuleVersion, CreatedAt.
MessageDraft	Id, LeadId, Version, Channel, Subject, Body, Status, ApprovedBy, ApprovedAt, CreatedAt.
CrmActivity	Id, LeadId, Type (call/message/reply/note), Channel, Note, OccurredAt, FollowUpDueAt.
DoNotContact	Id, Value (phone, email or domain), Kind, Reason, CreatedAt. Unique on Value.
AuditLog	Id, Action, Detail, Actor, CreatedAt.
AppUser	Id, Email (unique), PasswordHash, Role (admin/viewer), CreatedAt.

Step 4: Create the DbContext (Data/AppDbContext.cs)

```
using AiClientHunter.Api.Domain;
using Microsoft.EntityFrameworkCore;
namespace AiClientHunter.Api.Data;

public class AppDbContext(DbContextOptions<AppDbContext> options) : DbContext(options)
{
    public DbSet<Lead> Leads => Set<Lead>();
    public DbSet<SearchJob> SearchJobs => Set<SearchJob>();
    public DbSet<SourceRecord> SourceRecords => Set<SourceRecord>();
    public DbSet<LeadEvidence> EvidenceLinks => Set<LeadEvidence>();
    public DbSet<LeadScore> LeadScores => Set<LeadScore>();
    public DbSet<MessageDraft> MessageDrafts => Set<MessageDraft>();
    public DbSet<CrmActivity> CrmActivities => Set<CrmActivity>();
    public DbSet<DoNotContact> DoNotContactList => Set<DoNotContact>();
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();
    public DbSet<AppUser> Users => Set<AppUser>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<Lead>().HasIndex(l => l.DedupeKey).IsUnique();
        b.Entity<Lead>().HasIndex(l => new { l.LeadType, l.Status });
        b.Entity<SourceRecord>().HasIndex(s => new { s.SourceName, s.SourceId }).IsUnique();
        b.Entity<DoNotContact>().HasIndex(d => d.Value).IsUnique();
        b.Entity<AppUser>().HasIndex(u => u.Email).IsUnique();
    }
}
```

Table names follow the DbSet names (Leads, SearchJobs, and so on). The snake_case names in Section 6 are the logical names.

Step 5: Register it in Program.cs

```
using AiClientHunter.Api.Data;
using Microsoft.EntityFrameworkCore;

var cs = builder.Configuration.GetConnectionString("Default")
         ?? throw new InvalidOperationException("ConnectionStrings:Default is missing");
builder.Services.AddDbContext<AppDbContext>(o => o.UseMySql(cs, ServerVersion.AutoDetect(cs)));

// health check now also tests the database
app.MapGet("/api/health", async (AppDbContext db) => Results.Ok(new
{
    app = "AI Client Hunter API", status = "ok",
    database = await db.Database.CanConnectAsync() ? "ok" : "down"
}));
```

Step 6: Create and apply the migration

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

In Workbench, refresh the schema. You should see the tables plus __EFMigrationsHistory.

Step 7: Seed sample data and run a CRUD check

Add a Development-only seeder and a test endpoint. This proves create, read, update and delete work.

```
// Data/DbSeeder.cs
public static class DbSeeder
{
    public static async Task SeedAsync(AppDbContext db)
    {
        if (await db.Leads.AnyAsync()) return;
        db.Leads.AddRange(
            new Lead { Name = "Sample Clinic A", Category = "clinic", City = "Ahmedabad",
                       State = "Gujarat", WebsiteStatus = "not_found", LeadType = "potential",
                       Score = 20, DedupeKey = "sample a|ahmedabad" },
            new Lead { Name = "Sample Startup B", Category = "software", City = "Surat",
                       State = "Gujarat", WebsiteStatus = "unknown", LeadType = "warm",
                       Score = 40, DedupeKey = "sample b|surat" },
            new Lead { Name = "Need website for my shop", Category = "retail", City = "Rajkot",
                       State = "Gujarat", WebsiteStatus = "unknown", LeadType = "hot",
                       Score = 85, DedupeKey = "sample c|rajkot" });
        await db.SaveChangesAsync();
    }
}

// Program.cs, after app is built
if (app.Environment.IsDevelopment())
{
    using var scope = app.Services.CreateScope();
    await DbSeeder.SeedAsync(scope.ServiceProvider.GetRequiredService<AppDbContext>());

    app.MapPost("/api/dev/db-check", async (AppDbContext db) =>
    {
        var lead = new Lead { Name = "DB check", DedupeKey = "dbcheck|" + Guid.NewGuid() };
        db.Leads.Add(lead);              await db.SaveChangesAsync();   // create
        var read = await db.Leads.FindAsync(lead.Id);                   // read
        read!.Status = "needs_review";   await db.SaveChangesAsync();   // update
        db.Leads.Remove(read);           await db.SaveChangesAsync();   // delete
        return Results.Ok("create/read/update/delete passed");
    });
}
```

Step 8: Test persistence

Run `Invoke-RestMethod -Method Post http://localhost:5000/api/dev/db-check`. Then stop and restart the API (and MySQL) and confirm the 3 sample leads are still in the Leads table.

Completion check

- [ ] Migration applied and all tables visible in Workbench.
- [ ] /api/health shows database ok.

- [ ] db-check endpoint passes; sample leads survive a restart.
- [ ] The connection string is in user-secrets, not in any file in git.

Phase 3: CRM Basics

Goal	Lead list with filters, a detail page, status changes, a do-not-contact action and CSV export, all working with sample data.
Time (rough)	1 to 2 days

Before you start

- Phase 2 complete (sample leads in MySQL).
- Backend package: `dotnet add package CsvHelper`.
- Frontend package: `npm install react-router-dom`.

Step 1: Add the lead endpoints (Endpoints/LeadEndpoints.cs)

```
public static class LeadEndpoints
{
    static readonly HashSet<string> Statuses = new() { "new", "needs_review", "qualified", "draft_ready",
        "approved", "contacted", "replied", "follow_up_due", "won", "lost", "do_not_contact" };

    public static void MapLeadEndpoints(this WebApplication app)
    {
        var g = app.MapGroup("/api/leads");

        g.MapGet("", async (AppDbContext db, string? priority, string? status, string? q,
                            int page = 1, int pageSize = 25) =>
        {
            var query = db.Leads.AsNoTracking().AsQueryable();
            if (!string.IsNullOrWhiteSpace(priority)) query = query.Where(l => l.LeadType == priority);
            if (!string.IsNullOrWhiteSpace(status))   query = query.Where(l => l.Status == status);
            if (!string.IsNullOrWhiteSpace(q))
                query = query.Where(l => l.Name.Contains(q) || (l.City ?? "").Contains(q));

            var total = await query.CountAsync();
            var items = await query.OrderByDescending(l => l.Score).ThenByDescending(l => l.CreatedAt)
                                   .Skip((page - 1) * pageSize).Take(pageSize).ToListAsync();
            return Results.Ok(new { total, page, pageSize, items });
        });

        g.MapGet("{id:long}", async (long id, AppDbContext db) =>
            await db.Leads.Include(l => l.Evidence).AsNoTracking().FirstOrDefaultAsync(l => l.Id == id)
                is { } lead ? Results.Ok(lead) : Results.NotFound());

        g.MapPatch("{id:long}/status", async (long id, StatusUpdate body, AppDbContext db) =>
        {
            if (!Statuses.Contains(body.Status)) return Results.BadRequest("Unknown status");
            var lead = await db.Leads.FindAsync(id);
            if (lead is null) return Results.NotFound();
            lead.Status = body.Status; lead.UpdatedAt = DateTime.UtcNow;
            await db.SaveChangesAsync();
            return Results.Ok(lead);
        });
    }
}
public record StatusUpdate(string Status);
```

Call `app.MapLeadEndpoints();` in Program.cs. Test in the browser: `/api/leads?priority=hot`.

Step 2: Add manual lead creation and do-not-contact

POST /api/leads creates a lead you found by hand (for example from LinkedIn or an Upwork alert) with a source link in the note, so manual leads also have evidence. **POST /api/leads/{id}/do-not-contact** adds the lead's phone and domain to the DoNotContact table and sets the status to do_not_contact. Later phases check this table before any draft is created or approved.

Step 3: Add CSV export

```
g.MapGet("export", async (AppDbContext db, string? priority) =>
{
    var q = db.Leads.AsNoTracking().AsQueryable();
    if (!string.IsNullOrWhiteSpace(priority)) q = q.Where(l => l.LeadType == priority);
    var rows = await q.Select(l => new { l.Name, l.Category, l.City, l.Phone, l.WebsiteStatus,
                                         l.LeadType, l.Score, l.Status }).ToListAsync();
    using var sw = new StringWriter();
    using (var csv = new CsvHelper.CsvWriter(sw, System.Globalization.CultureInfo.InvariantCulture))
        csv.WriteRecords(rows);
    var bom = new System.Text.UTF8Encoding(true).GetPreamble();      // so Excel shows Gujarati/Hindi text
    var bytes = bom.Concat(System.Text.Encoding.UTF8.GetBytes(sw.ToString())).ToArray();
    return Results.File(bytes, "text/csv", "leads.csv");
});
```

Later (Phase 8) the export also includes the source link and approval status.

Step 4: Build the React pages

Wrap the app in a router (`BrowserRouter`) with routes `/` (lead list) and `/leads/:id` (detail). Start with one small helper for all API calls:

```
// client/src/api.js
const BASE = import.meta.env.VITE_API_BASE_URL ?? "";

export async function api(path, options = {}) {
  const res = await fetch(`${BASE}${path}`, {
    ...options,
    headers: { "Content-Type": "application/json", ...(options.headers || {}) },
  });
  if (!res.ok) throw new Error((await res.text()) || `HTTP ${res.status}`);
  return res.status === 204 ? null : res.json();
}
```

- **LeadList:** a table (name, city, priority badge, website status, score, status), filter bar (priority, status, search box), pagination, and an Export CSV button linking to /api/leads/export.
- **LeadDetail:** all fields, the evidence list with source links, a status dropdown that calls PATCH, and a Do not contact button with a confirmation.
- Show an "OpenStreetMap contributors" attribution line in the footer once OSM data is used (Phase 5).

Completion check

- [ ] Filters (priority, status, search) return the right rows.
- [ ] Status change on the detail page persists after refresh.
- [ ] A lead marked Do not contact adds a DoNotContact row.
- [ ] Exported CSV opens in Excel with correct text and the right columns.

Phase 4: Search Jobs and Background Worker

Goal	A command form creates a Search Job. A background worker runs it without freezing the UI, and the job's status and errors are visible.
Time (rough)	1 to 2 days

Before you start

- Phase 3 complete.
- No new packages. The queue uses .NET's built-in System.Threading.Channels and BackgroundService.
- Design rule: the database is the source of truth. The in-memory queue only holds job IDs, so unfinished jobs are re-queued after a restart.

Step 1: Create the queue and worker (Jobs/)

```
// JobQueue.cs
using System.Threading.Channels;
public class JobQueue
{
    private readonly Channel<long> _ch = Channel.CreateUnbounded<long>();
    public ValueTask EnqueueAsync(long jobId) => _ch.Writer.WriteAsync(jobId);
    public IAsyncEnumerable<long> ReadAllAsync(CancellationToken ct) => _ch.Reader.ReadAllAsync(ct);
}

// JobWorker.cs
public class JobWorker(IServiceScopeFactory scopes, JobQueue queue, ILogger<JobWorker> log) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        using (var scope = scopes.CreateScope())       // re-queue unfinished jobs after a restart
        {
            var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
            var ids = await db.SearchJobs.Where(j => j.Status == "queued" || j.Status == "running")
                                         .Select(j => j.Id).ToListAsync(ct);
            foreach (var id in ids) await queue.EnqueueAsync(id);
        }
        await foreach (var id in queue.ReadAllAsync(ct))   // one job at a time
        {
            try { await RunAsync(id, ct); }
            catch (Exception ex) { log.LogError(ex, "Job {Id} crashed", id); }
        }
    }

    private async Task RunAsync(long id, CancellationToken ct)
    {
        using var scope = scopes.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        var job = await db.SearchJobs.FindAsync(new object[] { id }, ct);
        if (job is null) return;
        job.Status = "running"; job.StartedAt = DateTime.UtcNow; job.Error = null;
        await db.SaveChangesAsync(ct);
        try
        {
            // Phase 5 and 6 plug in here: fetch, normalize, dedupe, verify, score
            job.Status = "completed";
        }
        catch (Exception ex)
        {
            job.Status = "failed"; job.Error = ex.Message;      // visible in the UI
        }
        job.FinishedAt = DateTime.UtcNow;
        await db.SaveChangesAsync(ct);
    }
}
```

Step 2: Register and add the endpoints

```
// Program.cs
builder.Services.AddSingleton<JobQueue>();
builder.Services.AddHostedService<JobWorker>();

// Endpoints/JobEndpoints.cs
var g = app.MapGroup("/api/search-jobs");

g.MapPost("", async (SearchJobRequest req, AppDbContext db, JobQueue queue) =>
{
    var errors = new Dictionary<string, string[]>();
    if (string.IsNullOrWhiteSpace(req.City))     errors["city"] = new[] { "City is required" };
    if (string.IsNullOrWhiteSpace(req.Category)) errors["category"] = new[] { "Category is required" };
    if (req.Limit is < 1 or > 200)               errors["limit"] = new[] { "Limit must be 1 to 200" };
    if (req.LeadType is not ("hot" or "warm" or "potential"))
                                                 errors["leadType"] = new[] { "Unknown lead type" };
    if (errors.Count > 0) return Results.ValidationProblem(errors);

    var job = new SearchJob { City = req.City.Trim(), Category = req.Category.Trim(),
                              LeadType = req.LeadType, ResultLimit = req.Limit };
    db.SearchJobs.Add(job);
    await db.SaveChangesAsync();
    await queue.EnqueueAsync(job.Id);
    return Results.Created($"/api/search-jobs/{job.Id}", job);
});

g.MapGet("", async (AppDbContext db) =>
    await db.SearchJobs.AsNoTracking().OrderByDescending(j => j.Id).Take(50).ToListAsync());

g.MapPost("{id:long}/retry", async (long id, AppDbContext db, JobQueue queue) =>
{
    var job = await db.SearchJobs.FindAsync(id);
    if (job is null) return Results.NotFound();
    if (job.Status != "failed") return Results.BadRequest("Only failed jobs can be retried");
    job.Status = "queued"; job.Error = null;
    await db.SaveChangesAsync();
    await queue.EnqueueAsync(id);
    return Results.Ok(job);
});

public record SearchJobRequest(string City, string Category, string LeadType, int Limit);
```

Retries are safe because Phase 6 deduplication merges repeated results instead of creating duplicates.

Step 3: Add the Search page in React

A form with city (dropdown of supported cities), category (dropdown), lead type and limit. On submit, POST to /api/search-jobs. Below the form, a jobs table (status badge, records found, new leads, duplicates, error text, a Retry button for failed jobs) that refreshes every 3 seconds with `setInterval` while any job is queued or running.

Completion check

- [ ] An empty city or an out-of-range limit shows a clear validation message.
- [ ] A valid job moves queued, running, completed in the table without freezing the page.
- [ ] Throw a test exception in RunAsync: the job shows failed with the error, and Retry works.
- [ ] Restart the API while a job is queued: it is picked up again.

Phase 5: First Source: OpenStreetMap

Goal	A search job fetches real businesses from OpenStreetMap, and every result keeps its source URL and retrieval time.
Time (rough)	2 to 3 days

Before you start

- Phase 4 complete, and the Phase 0 test shows OSM has usable data for your city and category.
- No new packages. Use HttpClient.
- Overpass fair use: one small query per job, a descriptive User-Agent with your contact, a 24-hour cache, and no retry loops. The public server guidance is about 10,000 requests and 1 GB per day, and heavy users are throttled first.

Step 1: Define the adapter interface (Sources/ISourceAdapter.cs)

```
public record SearchRequest(string City, string Category, int Limit);

public record RawRecord(string SourceName, string SourceId, string SourceUrl, string Name,
    string? Category, string? City, string? Phone, string? Website, string RawJson, DateTime RetrievedAt);

public interface ISourceAdapter
{
    string Name { get; }
    Task<IReadOnlyList<RawRecord>> FetchAsync(SearchRequest req, CancellationToken ct);
}
```

Step 2: Map cities and categories (Sources/OsmMaps.cs)

```
public static class OsmMaps
{
    // bbox = south,west,north,east (approximate; check each box on a map)
    public static readonly Dictionary<string, string> CityBoxes = new(StringComparer.OrdinalIgnoreCase)
    {
        ["Ahmedabad"]   = "22.90,72.45,23.15,72.75",
        ["Gandhinagar"] = "23.15,72.55,23.35,72.75",
        ["Surat"]       = "21.05,72.70,21.30,72.95",
        ["Vadodara"]    = "22.20,73.05,22.40,73.30",
        ["Rajkot"]      = "22.20,70.70,22.35,70.90",
    };

    public static readonly Dictionary<string, string> CategoryFilters = new(StringComparer.OrdinalIgnoreCase)
    {
        ["clinic"]     = "[\"amenity\"~\"^(clinic|doctors|dentist)$\"]",
        ["restaurant"] = "[\"amenity\"~\"^(restaurant|cafe)$\"]",
        ["salon"]      = "[\"shop\"~\"^(hairdresser|beauty)$\"]",
        ["gym"]        = "[\"leisure\"=\"fitness_centre\"]",
        ["school"]     = "[\"amenity\"~\"^(school|kindergarten)$\"]",
        ["hotel"]      = "[\"tourism\"~\"^(hotel|guest_house)$\"]",
    };
}
```

Step 3: Write the Overpass adapter (Sources/OsmOverpassAdapter.cs)

```
public class OsmOverpassAdapter(HttpClient http) : ISourceAdapter
{
    public string Name => "osm";

    public async Task<IReadOnlyList<RawRecord>> FetchAsync(SearchRequest req, CancellationToken ct)
    {
        if (!OsmMaps.CityBoxes.TryGetValue(req.City, out var bbox))
            throw new InvalidOperationException($"City not supported yet: {req.City}");
        if (!OsmMaps.CategoryFilters.TryGetValue(req.Category, out var filter))
            throw new InvalidOperationException($"Category not supported yet: {req.Category}");

        var query = $"[out:json][timeout:60][bbox:{bbox}];nwr{filter};out center tags meta;";
        using var body = new FormUrlEncodedContent(new[] { new KeyValuePair<string, string>("data", query) });
        using var resp = await http.PostAsync("https://overpass-api.de/api/interpreter", body, ct);
        resp.EnsureSuccessStatusCode();                    // 429 or 504 = wait, do not hammer
        await using var stream = await resp.Content.ReadAsStreamAsync(ct);
        using var doc = await JsonDocument.ParseAsync(stream, cancellationToken: ct);

        var results = new List<RawRecord>();
        foreach (var el in doc.RootElement.GetProperty("elements").EnumerateArray())
        {
            if (!el.TryGetProperty("tags", out var tags)) continue;
            string? Tag(string k) => tags.TryGetProperty(k, out var v) ? v.GetString() : null;

            var name = Tag("name");
            if (string.IsNullOrWhiteSpace(name)) continue;            // unnamed places are not useful leads
            var type = el.GetProperty("type").GetString()!;
            var id = el.GetProperty("id").GetInt64();
            results.Add(new RawRecord(
                "osm", $"{type}/{id}", $"https://www.openstreetmap.org/{type}/{id}", name,
                req.Category, req.City,
                Tag("phone") ?? Tag("contact:phone") ?? Tag("contact:mobile"),
                Tag("website") ?? Tag("contact:website") ?? Tag("url"),
                el.GetRawText(), DateTime.UtcNow));
        }
        return results;
    }
}

// Program.cs
builder.Services.AddHttpClient<ISourceAdapter, OsmOverpassAdapter>(c =>
{
    c.Timeout = TimeSpan.FromSeconds(90);
    c.DefaultRequestHeaders.UserAgent.ParseAdd("AiClientHunter/0.1 (contact: you@example.com)"); // use your email
});
```

Step 4: Cache and protect the public server

Before calling Overpass, look in SourceRecords for the same city and category fetched in the last 24 hours and reuse it. Run one job at a time (already true with the single worker). If you get HTTP 429 or 504, mark the job failed with a clear message and let the user retry later. For bulk or repeated needs, download the Geofabrik India data instead (for example `western-zone-latest.osm.pbf`, about 210 MB; check the zone page to confirm which zone covers your state) and process it locally. That is an optional later upgrade.

Step 5: Be honest about "new business" signals

OpenStreetMap does not say when a business opened. Two weak signals exist: a `start_date` tag (rare), and an object that is at version 1 with a recent timestamp, which only means it was recently *added to the map*. Treat the second as a small bonus (about +10), not as a Warm signal on its own.

Step 6: Wire it into the worker

In JobWorker.RunAsync call the adapter, keep records whose website is empty when the job's lead type is "website missing", save each as a SourceRecord with its URL and RetrievedAt, and set job.RecordsFound. Show the attribution line "Data from OpenStreetMap contributors (ODbL)" in the dashboard footer.

Completion check

- [ ] A real job for Ahmedabad clinics returns records with source URLs and timestamps.

- [ ] Running the same job twice within 24 hours makes no second Overpass call (check the log).
- [ ] A forced 429 or timeout shows a readable error and the job can be retried.
- [ ] The attribution line is visible in the app.

Phase 6: Verification, Deduplication and Scoring

Goal	Records are normalized, merged into unique leads, website status is checked honestly, and each lead gets a rule-based score with reasons.
Time (rough)	3 to 5 days

Before you start

- Phase 5 complete (raw records arrive in the worker).
- Rules to keep: never guess. A domain that merely resolves is only a *candidate*, not a verified website. Not found means "our checks found nothing", not "no website exists".

Step 1: Normalize data (Services/Normalizer.cs)

```
public static class Normalizer
{
    public static string? Phone(string? raw)              // returns +91XXXXXXXXXX or null
    {
        if (string.IsNullOrWhiteSpace(raw)) return null;
        var first = raw.Split(';', ',', '/')[0];
        var d = new string(first.Where(char.IsDigit).ToArray());
        if (d.Length == 12 && d.StartsWith("91")) d = d[2..];
        else if (d.Length == 11 && d.StartsWith("0")) d = d[1..];
        return d.Length == 10 ? "+91" + d : null;
    }

    public static string? Domain(string? url)
    {
        if (string.IsNullOrWhiteSpace(url)) return null;
        if (!url.Contains("://")) url = "http://" + url;
        if (!Uri.TryCreate(url, UriKind.Absolute, out var u)) return null;
        var h = u.Host.ToLowerInvariant();
        return h.StartsWith("www.") ? h[4..] : h;
    }

    // keeps letters and digits of any language (Gujarati, Hindi, English)
    public static string NameKey(string s) =>
        System.Text.RegularExpressions.Regex.Replace(s.ToLowerInvariant(), @"[^\p{L}\p{N}]+", " ").Trim();
}
```

Step 2: Deduplicate (Services/DedupeService.cs)

Check in this order and stop at the first match:

- Same SourceName + SourceId already linked to a lead: update that lead's evidence.
- Same normalized domain: same business.
- Same normalized name + same city (and same phone if both have one).
- Otherwise create a new lead.

```
// DedupeKey stored on every lead: domain if known, otherwise name|city
var key = domain ?? $"{Normalizer.NameKey(r.Name)}|{Normalizer.NameKey(r.City ?? "")}";

var existing =
    await db.EvidenceLinks.Where(e => e.SourceRecord.SourceName == r.SourceName
                                   && e.SourceRecord.SourceId == r.SourceId)
                          .Select(e => e.Lead).FirstOrDefaultAsync(ct)
    ?? await db.Leads.FirstOrDefaultAsync(l => l.DedupeKey == key, ct);

if (existing is null) { /* create Lead with DedupeKey = key, count as new */ }
else                  { /* add evidence if missing, bump UpdatedAt, count as duplicate */ }
```

Step 3: Check websites (Services/WebsiteChecker.cs)

```
public record WebsiteCheckResult(string Status, string? Url, string Note);

public class WebsiteChecker(HttpClient http)       // HttpClient: 10 second timeout, User-Agent set
{
    // The listing already has a URL: does it respond?
    public async Task<WebsiteCheckResult> CheckListedAsync(string url, CancellationToken ct)
    {
        try
        {
            using var resp = await http.GetAsync(url, HttpCompletionOption.ResponseHeadersRead, ct);
            return (int)resp.StatusCode < 400
                ? new("verified", url, $"HTTP {(int)resp.StatusCode}")
                : new("unknown", url, $"HTTP {(int)resp.StatusCode}");
        }
        catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
        {
            return new("unknown", url, ex.GetType().Name);          // could be temporary, so not "not_found"
        }
    }

    // No URL in the listing: try a few likely domains with DNS only
    public async Task<WebsiteCheckResult> FindCandidateAsync(string businessName, CancellationToken ct)
    {
        var slug = new string(businessName.ToLowerInvariant()
            .Where(c => c is (>= 'a' and <= 'z') or (>= '0' and <= '9')).ToArray());
        if (slug.Length < 4) return new("unknown", null, "Name too short or not in Latin letters");
        foreach (var host in new[] { slug + ".com", slug + ".in", slug + ".co.in" })
        {
            try
            {
                var ips = await System.Net.Dns.GetHostAddressesAsync(host, ct);
                if (ips.Length > 0)
                    return new("unknown", "https://" + host,
                               "Domain exists. Confirm manually it belongs to this business.");
            }
            catch (System.Net.Sockets.SocketException) { /* does not resolve, try next */ }
        }
        return new("not_found", null,
                   "No likely domain resolved. This is not proof that no website exists.");
    }
}
```

Run checks with limited parallelism (for example `SemaphoreSlim(5)`), never hit the same host in a tight loop, and only re-check a lead when WebsiteCheckedAt is older than a few days.

Step 4: Score with rules (Services/Scorer.cs)

This is the table from Section 2 turned into code. Save the reasons so the UI can show why a lead got its priority.

```
public record LeadSignals(bool ExplicitRequest, bool RequestDetails, bool Recent7Days, bool NewBusinessSignal,
    bool WebsiteFieldMissing, bool NoLiveSiteFound, bool HasPhone, bool InTargetArea);

public record ScoreResult(int Points, string Priority, List<string> Reasons);

public static class Scorer
{
    public static ScoreResult Score(LeadSignals s)
    {
        int pts = 0; var why = new List<string>();
        void Add(bool cond, int p, string text) { if (cond) { pts += p; why.Add($"+{p} {text}"); } }

        Add(s.ExplicitRequest,     60, "Public post asks for a website/app/software developer");
        Add(s.RequestDetails,      10, "Post mentions budget, deadline or details");
        Add(s.Recent7Days,         10, "Posted or listed in the last 7 days");
        Add(s.NewBusinessSignal,   25, "New-business signal");
        Add(s.WebsiteFieldMissing, 15, "Website missing in the source listing");
        Add(s.NoLiveSiteFound,     10, "No live site found by our checks");
        Add(s.HasPhone,             5, "Phone available from a permitted source");
        Add(s.InTargetArea,         5, "In a target city");

        var priority = s.ExplicitRequest && pts >= 60 ? "hot" : pts >= 30 ? "warm" : "potential";
        return new ScoreResult(pts, priority, why);
    }
}
```

Step 5: Put the pipeline together in the worker

```
foreach (var r in records.Take(job.ResultLimit))
{
    var phone  = Normalizer.Phone(r.Phone);
    var domain = Normalizer.Domain(r.Website);
    // 1. find or create the lead (DedupeService), 2. save SourceRecord + LeadEvidence
    // 3. website check: domain is null ? FindCandidateAsync : CheckListedAsync
    // 4. build LeadSignals, call Scorer.Score, store Score, LeadType and a LeadScore row (RuleVersion "v1")
    // 5. SaveChanges, update job.NewLeads / job.Duplicates
}
```

Completion check

- [ ] Running the same job twice gives NewLeads = 0 and Duplicates = the record count.
- [ ] Phone numbers are stored as +91XXXXXXXXXX or empty. Gujarati names stay intact.
- [ ] A listing with a live site, a dead domain, and no site each get the right status.
- [ ] Every lead shows its score reasons and a source link.
- [ ] Nothing is marked Hot unless an explicit request exists.

Phase 7: AI Analysis with Ollama

Goal	A local model summarizes a lead from its evidence and suggests a service, returning structured JSON that never invents facts.
Time (rough)	3 to 4 days

Before you start

- Install Ollama for Windows from ollama.com. Then pull a small model: `ollama pull llama3.2:3b` (light, works on 8 GB RAM). For better quality with 16 GB RAM try `ollama pull qwen2.5:7b`.
- Test it: `ollama run llama3.2:3b`, say hello, then exit. Ollama listens on http://localhost:11434.
- On a laptop CPU one answer can take 10 to 60 seconds. So run the AI only on demand or for Hot and Warm leads, never in bulk over every lead.

Step 1: Put the AI behind an interface (Ai/IAiProvider.cs)

```
public record AiAnalysis(string Summary, string RecommendedService, string DraftMessage, string[] MissingInfo);

public interface IAiProvider
{
    Task<AiAnalysis> AnalyzeAsync(string evidenceText, CancellationToken ct);
}
```

Today OllamaProvider implements it. A hosted model later is one new class plus a config switch.

Step 2: Write the Ollama provider (Ai/OllamaProvider.cs)

```
public class OllamaProvider(HttpClient http, IConfiguration cfg) : IAiProvider
{
    const string SystemPrompt =
        "You help a freelance web developer write short, polite first messages. " +
        "Use ONLY the facts in the evidence. If a fact is not there, do not mention it " +
        "and list it in missing_info. " +
        "Never invent phone numbers, prices, names, links or claims about the business. " +
        "Draft: 3 to 5 sentences, no hype, say honestly how the lead was found, " +
        "and include a polite way to opt out.";

    static readonly object Schema = new
    {
        type = "object",
        properties = new
        {
            summary = new { type = "string" },
            recommended_service = new
            {
                type = "string",
                @enum = new[] { "website", "mobile_app", "software", "automation", "unknown" }
            },
            draft_message = new { type = "string" },
            missing_info = new { type = "array", items = new { type = "string" } }
        },
        required = new[] { "summary", "recommended_service", "draft_message", "missing_info" }
    };

    public async Task<AiAnalysis> AnalyzeAsync(string evidenceText, CancellationToken ct)
    {
        var body = new
        {
            model = cfg["Ollama:Model"] ?? "llama3.2:3b",
            stream = false,
            format = Schema,                                    // forces JSON that matches the schema
            options = new { temperature = 0.2 },
            messages = new object[]
            {
                new { role = "system", content = SystemPrompt },
                new { role = "user",   content = evidenceText }
            }
        };
        var baseUrl = cfg["Ollama:BaseUrl"] ?? "http://localhost:11434";
        using var resp = await http.PostAsJsonAsync($"{baseUrl}/api/chat", body, ct);
        resp.EnsureSuccessStatusCode();
        var json = await resp.Content.ReadFromJsonAsync<JsonElement>(cancellationToken: ct);
        var content = json.GetProperty("message").GetProperty("content").GetString()!;
        var opts = new JsonSerializerOptions { PropertyNamingPolicy = JsonNamingPolicy.SnakeCaseLower };
        return JsonSerializer.Deserialize<AiAnalysis>(content, opts)
               ?? throw new InvalidOperationException("Empty AI response");
    }
}

// Program.cs  (slow laptops need a long timeout)
builder.Services.AddHttpClient<IAiProvider, OllamaProvider>(c => c.Timeout = TimeSpan.FromMinutes(3));
```

Step 3: Build the evidence text

Give the model only collected facts, as plain lines. Example:

```
Business: Shah Dental Clinic
Category: clinic
City: Ahmedabad, Gujarat
Website status: not_found (checked 2026-10-14; no listed site, no likely domain resolved)
Phone available: yes
Lead source: OpenStreetMap node/123456789 (retrieved 2026-10-14)
Signals: website missing in listing
```

Step 4: Add guardrails after every AI call

Never trust the model blindly. Reject and retry once (then mark "ai_failed") if any of these is true: a field is empty, recommended_service is not in the list, or the draft contains a URL or phone number that is not in the evidence.

```
static readonly Regex Risky = new(@"https?://\S+|\+?\d[\d\s-]{8,}\d", RegexOptions.Compiled);

public static bool MentionsOnlyKnownFacts(string draft, string evidenceText) =>
    Risky.Matches(draft).All(m => evidenceText.Contains(m.Value.Trim(), StringComparison.OrdinalIgnoreCase));
```

Step 5: Test with 10 hand-written cases

Create 10 evidence texts and run them all. Include: a lead with almost no data, one with no phone, one in Gujarati, one with a clear project request, and one that tempts the model to invent a price. Pass condition: no invented phone, link, price or fact in any output, and missing_info is filled when data is missing. If a 3B model fails often, try the 7B model or shorten the prompt.

Completion check

- [ ] Valid JSON comes back every time (schema enforced).
- [ ] None of the 10 test cases contains an invented fact.
- [ ] AI is called only on demand or for chosen leads, and a timeout or Ollama-off case shows a readable error.
- [ ] Switching the model name in appsettings changes the model without code changes.

Phase 8: Message Drafts and Approval Queue

Goal	Drafts are generated per lead, reviewed and edited by you, approved, then sent manually by you. No code path sends anything automatically.
Time (rough)	3 to 4 days

Before you start

- Phase 7 complete.
- Sending stays manual: WhatsApp click-to-chat link or your own email app. This avoids WhatsApp Business API approvals and SMS DLT costs.
- Be considerate even when sending by hand: keep volume small, make messages relevant, and honor any request to stop.

Step 1: Check the do-not-contact list first (Services/Suppression.cs)

```
public static class Suppression
{
    public static async Task<bool> IsBlockedAsync(AppDbContext db, Lead lead)
    {
        var domain = Normalizer.Domain(lead.WebsiteUrl);
        return await db.DoNotContactList.AnyAsync(d =>
            (lead.Phone != null && d.Value == lead.Phone) ||
            (domain != null && d.Value == domain));
    }
}
```

Step 2: Add the draft endpoints (Endpoints/DraftEndpoints.cs)

POST /api/leads/{id}/drafts/generate: block if suppressed, call the AI, save a MessageDraft with Version + 1 and status "draft". **PUT /api/drafts/{id}:** edit the body (an edit to an approved draft returns it to "draft" so it must be approved again). **GET /api/drafts?status=draft:** the approval queue, which skips suppressed leads. **POST /api/drafts/{id}/mark-sent:** only for approved drafts; records a CrmActivity, sets the lead to contacted and sets a follow-up date.

```
g.MapPost("drafts/{id:long}/approve", async (long id, AppDbContext db, ClaimsPrincipal user) =>
{
    var d = await db.MessageDrafts.Include(x => x.Lead).FirstOrDefaultAsync(x => x.Id == id);
    if (d is null) return Results.NotFound();
    if (d.Status != "draft") return Results.BadRequest("Only drafts can be approved");
    if (await Suppression.IsBlockedAsync(db, d.Lead))
        return Results.Conflict("Lead is on the do-not-contact list");

    d.Status = "approved";
    d.ApprovedBy = user.Identity?.Name ?? "local";
    d.ApprovedAt = DateTime.UtcNow;
    db.AuditLogs.Add(new AuditLog { Action = "draft_approved", Actor = d.ApprovedBy,
                                    Detail = $"draft {id} for lead {d.LeadId}" });
    await db.SaveChangesAsync();
    return Results.Ok(d);
});
```

Step 3: Add the Approval Queue page and send buttons

```
// client/src/pages/ApprovalQueue.jsx (helpers)
const waUrl = (phone, text) =>
  `https://wa.me/${phone.replace(/\D/g, "")}?text=${encodeURIComponent(text)}`;

const mailUrl = (email, subject, body) =>
  `mailto:${email}?subject=${encodeURIComponent(subject)}&body=${encodeURIComponent(body)}`;

// Show "Open WhatsApp" / "Open email" ONLY when draft.status === "approved".
// After you send manually, click "Mark as sent" to log it and set the follow-up date.
```

Step 4: Follow-ups and limits

Add a dashboard filter "Follow-up due" (FollowUpDueAt on or before today). Add a soft daily counter: show a warning when more than 20 messages were marked sent today (make the number a setting). Include an opt-out line in every draft and set the lead to do_not_contact the moment someone asks to stop.

Completion check

- [ ] No message can be marked sent unless the draft is approved.
- [ ] Editing an approved draft sends it back to draft status.
- [ ] A lead on the do-not-contact list cannot get a draft generated or approved, and is not in the queue.
- [ ] wa.me and mailto links open with the correct pre-filled text; the AuditLogs table records approvals.
- [ ] Search the code: no HTTP call or SDK exists that sends a message to a customer.

Phase 9: Full Testing and Security

Goal	Automated tests for the risky logic, a manual checklist for the rest, login and roles on every endpoint, and a proven backup and restore.
Time (rough)	4 to 6 days

Before you start

- Phases 1 to 8 complete.
- Packages: xUnit test project, Microsoft.EntityFrameworkCore.Sqlite (for tests), BCrypt.Net-Next, Microsoft.AspNetCore.Authentication.JwtBearer, System.IdentityModel.Tokens.Jwt.

Step 1: Create the test project

```
cd server
dotnet new xunit -n AiClientHunter.Tests
dotnet add AiClientHunter.Tests reference AiClientHunter.Api
dotnet add AiClientHunter.Tests package Microsoft.EntityFrameworkCore.Sqlite
dotnet test
```

Step 2: Write unit tests for the logic that can hurt you

```
public class NormalizerTests
{
    [Theory]
    [InlineData("+91 98765 43210", "+919876543210")]
    [InlineData("098765 43210",    "+919876543210")]
    [InlineData("12345",           null)]
    public void Phone_is_normalized(string raw, string? expected) =>
        Assert.Equal(expected, Normalizer.Phone(raw));

    [Fact]
    public void Domain_strips_scheme_www_and_path() =>
        Assert.Equal("shahclinic.in", Normalizer.Domain("https://www.ShahClinic.in/contact"));
}

public class ScorerTests
{
    [Fact]
    public void No_explicit_request_is_never_hot()
    {
        var s = new LeadSignals(false, true, true, true, true, true, true, true);
        Assert.NotEqual("hot", Scorer.Score(s).Priority);
    }
}
```

Also test: deduplication with SQLite in memory (same record twice gives one lead), the do-not-contact check, and that an unapproved draft cannot be marked sent.

Step 3: Run the manual test checklist

Tick each line with the exact action shown.

Area	How to test	Pass
Command validation	Submit empty city, limit 0, limit 500.	Clear message each time
Source adapter	Run the same job twice in a row; trigger a timeout.	Second run uses cache; error is readable
Deduplication	Run one job twice.	NewLeads 0 on the second run
Website check	Test a live site, a dead domain, a place with no site.	Three correct statuses
AI output	Run the 10 evidence cases.	No invented phone, link or price
Persistence	Restart API and MySQL.	All data still present

Area	How to test	Pass
Filters and export	Filter Hot, export, open in Excel.	Only Hot rows; text not garbled
Approval flow	Try mark-sent on an unapproved draft.	Rejected
Privacy and opt-out	Add a phone to do-not-contact, generate a draft.	Blocked; not in queue
Security	Call /api/leads with no token.	401
Performance	Start 3 jobs; kill MySQL mid-job; retry.	UI stays responsive; job fails clearly; retry works

Step 4: Add login and roles

Create a login endpoint that checks a BCrypt password hash and returns a short-lived JWT (a few hours). Put the signing key in user-secrets (`Jwt:Key`, at least 32 random characters). Create the first admin from configuration on first run when no users exist. Protect every group with `RequireAuthorization()` and make export and do-not-contact removal admin only. Add a rate limit on the login endpoint. In React, keep the token in memory or sessionStorage and send it as a Bearer header from api.js.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o => o.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = false, ValidateAudience = false,      // set issuer and audience for production
        ValidateLifetime = true,
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!))
    });
builder.Services.AddAuthorization();
// after app is built
app.UseAuthentication();
app.UseAuthorization();
app.MapGroup("/api/leads").RequireAuthorization();
```

Step 5: Security and dependency checks

```
git grep -nEi "password=|apikey|api_key|secret|token" -- . ":!*.md" ":!*.lock"
dotnet list package --vulnerable
cd client
npm audit
dotnet dev-certs https --trust
```

Also open client/dist after a build and confirm no keys or passwords appear in it.

Step 6: Backup and restore test

```
mysqldump -u hunter -p ai_client_hunter > backup.sql
mysql -u root -p -e "CREATE DATABASE ai_client_hunter_restore CHARACTER SET utf8mb4"
mysql -u root -p ai_client_hunter_restore < backup.sql
```

Compare row counts of Leads in both databases, then drop the restore database. A backup that has never been restored is not a backup.

Completion check

- [ ] All automated tests pass (dotnet test).
- [ ] Every line of the manual checklist passes.
- [ ] No endpoint except login and health works without a token.
- [ ] Secret scan is clean; no key in the frontend build.
- [ ] Restore test succeeded with matching row counts.

Phase 10: Deployment

Goal	The app runs securely online with HTTPS, backups, monitoring and a rollback plan, and passes a smoke test.
Time (rough)	2 to 4 days

Before you start

- Phase 9 complete. Do not deploy before login, tests and the restore test pass.
- Hosting is the first real cost of the project. Local-only use stays Rs. 0. Check current provider prices before choosing.
- Ollama needs a strong machine. A small server usually cannot run it, so online the AI is either off (generate drafts on your laptop) or uses a paid hosted model through the IAiProvider interface.

Step 1: Choose a hosting route

Option	Good for	Notes
A. Keep it on your laptop, reach it privately	Solo use, lowest cost	Use a private tunnel or VPN tool for access from other devices. Check each tool's free-tier terms.
B. Small Linux VPS (recommended for going online)	One user or a small team	Runs Nginx, the API and MySQL together on one machine. You manage updates and backups.
C. Managed app and database services	Less server upkeep	Usually costs more. Check pricing and MySQL support first.

Step 2: Build the release

```
dotnet publish server/AiClientHunter.Api -c Release -o publish/api

cd client
echo VITE_API_BASE_URL= > .env.production      (empty = same origin, no CORS needed)
npm run build                                    (output: client/dist)
```

Step 3: Server setup (Ubuntu, route B)

Install the .NET 10 runtime (ASP.NET Core), Nginx, MySQL and certbot. Create the database and a dedicated MySQL user. Make MySQL listen on localhost only (`bind-address = 127.0.0.1`). Open only ports 22, 80 and 443 in the firewall. Copy `publish/api` to `/var/www/hunter/api` and `client/dist` to `/var/www/hunter/web`.

Step 4: Run the API as a service

```
# /etc/systemd/system/hunter-api.service
[Unit]
Description=AI Client Hunter API
After=network.target mysql.service

[Service]
WorkingDirectory=/var/www/hunter/api
ExecStart=/usr/bin/dotnet /var/www/hunter/api/AiClientHunter.Api.dll
Restart=always
User=www-data
Environment=ASPNETCORE_ENVIRONMENT=Production
EnvironmentFile=/etc/hunter/hunter.env

[Install]
WantedBy=multi-user.target

# /etc/hunter/hunter.env   (chmod 600, owned by root)
Urls=http://127.0.0.1:5000
ConnectionStrings__Default=Server=localhost;Database=ai_client_hunter;User=hunter;Password=...
Jwt__Key=...at least 32 random characters...
```

Enable it with `sudo systemctl enable --now hunter-api`.

Step 5: Nginx and HTTPS

```
server {
    server_name yourdomain.com;
    root /var/www/hunter/web;
    location /api/ { proxy_pass http://127.0.0.1:5000; proxy_set_header Host $host;
                     proxy_set_header X-Forwarded-Proto $scheme; }
    location /     { try_files $uri /index.html; }
}
# then:  sudo certbot --nginx -d yourdomain.com
```

Step 6: Apply database changes safely

Always take a backup first. Create a migration bundle on your laptop with `dotnet ef migrations bundle --self-contained -r linux-x64` and run the bundle on the server with the production connection string. Do not let the app migrate the database on its own at startup in production.

Step 7: Backups, monitoring and logs

```
# crontab -e   (daily at 2 AM; copy the files off the server too)
0 2 * * * mysqldump -u hunter -p'...' ai_client_hunter | gzip > /var/backups/hunter/$(date +\%F).sql.gz
```

Point a free uptime monitor at `/api/health` (it checks the database too). Read logs with `journalctl -u hunter-api`. Test a restore from a real backup once a month.

Step 8: Rollback plan (write it down before deploying)

- Keep the previous release folder (for example `api.prev`) and the previous web build.
- Take a database backup immediately before every migration.
- To roll back: stop the service, switch to the previous folders, restore the pre-migration backup if the schema changed, start the service, run the smoke test.

Step 9: Smoke test after every deploy

- https://yourdomain.com loads and shows the login page; HTTP redirects to HTTPS.
- /api/health returns ok with database ok; /api/leads without a token returns 401.
- Log in, run one small search job, see leads with source links, create a draft (or confirm AI is off online).
- Approve a draft: no message is sent anywhere; the audit log has an entry.
- Last backup file exists and is from today.

Completion check

- [ ] Site runs over HTTPS and all smoke-test lines pass.
- [ ] Secrets live only in /etc/hunter/hunter.env, not in the repo or the web folder.
- [ ] Daily backup runs and a restore from it was tested.
- [ ] Uptime monitor is active and the rollback steps are written down.

Appendix A: Troubleshooting

Problem	Likely cause and fix
dotnet ef: command not found	Run `dotnet tool install --global dotnet-ef`, then open a new terminal.
Unable to connect to any of the specified MySQL hosts	MySQL service is stopped, wrong port, or wrong user or password. Start the service in Windows Services and re-check the user-secrets connection string.
Specified key was too long (during migration)	An indexed string column has no length. Add [MaxLength(300)] or smaller to it and create a new migration.
Browser console shows a CORS error	Cors:AllowedOrigins must include http://localhost:5173. Restart the API after changing it.
Port 5000 already in use	Change Urls in appsettings.json and VITE_API_BASE_URL in client/.env to the same new port.
Overpass returns 429 or 504	Too many or too heavy queries. Wait, use a smaller bbox, keep the 24-hour cache, never retry in a loop.
Ollama connection refused	Start the Ollama app (or run `ollama serve`) and check the model with `ollama list`.
Ollama is very slow	Use the 3B model, generate drafts only on demand, close other heavy apps, raise the HTTP timeout.
CSV shows broken Gujarati or Hindi text in Excel	Write the UTF-8 BOM at the start of the file (done in the export code).
Login works but every call is 401	Token not sent, expired, or Jwt:Key differs between login and validation.

Appendix B: Master Checklist

Phase	Done when
0 Source test	Numbers for OSM recorded; keep or drop decided for each source.
1 Local setup	API and React run; React shows Connected; first commit made.
2 MySQL	Migration applied; CRUD check passes; data survives restart.
3 CRM basics	Filters, detail, status change, do-not-contact and CSV export work.
4 Search jobs	Jobs run in the background; errors visible; retry and restart recovery work.
5 First source	Real OSM results with source URLs and timestamps; cache and attribution in place.
6 Verify, dedupe, score	No duplicates on re-run; honest website statuses; scores show reasons.
7 AI analysis	Schema-valid JSON; 10 test cases without invented facts.
8 Drafts and approval	Nothing can be sent without approval; do-not-contact enforced; manual send links work.
9 Testing and security	Tests and checklist pass; login on all endpoints; restore test passed.
10 Deployment	HTTPS, backups, monitoring, rollback plan and smoke test complete.

Next step: start Phase 0 (about an hour, no code), then tell Claude **"Phase 2 code do"** to receive the full MySQL files and migrations. Each phase can be requested the same way, one at a time.